

SOC AI Rulebot MVP Build Guide and Executive Summary

Prepared for internal use. This document captures the currently working MVP architecture, the backend and frontend build process, deployment steps, testing approach, and the next modifications planned before release to the SOC team.

Note: I cannot save files directly into your SharePoint or OneDrive from here, but I have created this PDF so you can download it and upload it there.

1. Executive Summary (Manager-Friendly)

A working MVP of the SOC AI Rulebot has now been achieved using a modernized architecture. The final working pattern uses a Teams SDK-based frontend and an Azure Container Apps backend. This replaces earlier brittle approaches that depended on legacy bot and channel plumbing.

- **What works now:** Teams chat can send a rule ID (for example, 165072), receive QRadar rule analysis, ask general QRadar questions, and request a new offense analysis template.
- **Backend capabilities working:** deterministic rule analysis, generic reasoning, offense intake workflow, offense analysis scaffold, and local file/PDF-backed retrieval scaffolding for grounded responses.
- **Deployment model:** backend deployed to Azure Container Apps; frontend prepared as a separate Teams SDK project for Teams-facing runtime and packaging.
- **Why multiple path changes happened:** earlier development paths relied on Azure Bot / Bot Framework style integration and other older Microsoft app flows that proved brittle in practice. The current design is cleaner: Teams frontend runtime communicates directly with the backend service.
- **Immediate next work before SOC release:** improve retrieval quality, production hardening, frontend runtime deployment consistency, security cleanup, and broader offense-analysis enrichment.

Key Outcome

The most important milestone is that the project has crossed from prototype-only status into an end-to-end MVP with a real Teams user experience and a real deployed backend service.

2. Working MVP Architecture

The currently intended architecture is split into two independently deployable services:

- **Frontend:** Teams SDK application (Teams-only user interface).
- **Backend:** Rulebot API service hosted in Azure Container Apps.

User in Teams

- > Teams SDK frontend runtime
- > HTTPS POST to backend /message endpoint
- > Backend routes request:
 - numeric rule ID -> rule analysis
 - "new offense" -> offense template
 - filled offense template -> offense analysis
 - free text -> reasoning / RAG retrieval path
- > Frontend relays reply back to Teams

This separation is important: the Teams app package is not executable code. The Teams app package only describes the Teams app (manifest + icons). The live application logic must run in a real service over HTTPS.

3. Current MVP Features Confirmed Working

- **Rule ID analysis:** sending a pure numeric rule ID triggers deterministic lookup and structured analysis.

- **Generic reasoning:** free-text Teams or API messages trigger reasoning over the backend logic.
- **Offense intake:** “new offense” returns a structured template for the analyst to populate.
- **Offense analysis scaffold:** a completed template can be parsed and analysed with rule enrichment.
- **Backend deployment:** backend is deployed to Azure Container Apps and responds correctly via HTTPS.
- **Modern Teams path:** a new Teams SDK-based frontend project and Teams app package have been created, submitted, and approved for the new path.

4. Backend Build Guide (Current Working MVP)

4.1 Backend Repository Purpose

Backend repository: **soc-ai-rulebot-agent**. This repo contains the Rulebot intelligence and service logic. It is the source of truth for rule analysis, offense analysis, prompts, retrieval, and backend API routing.

4.2 Recommended Backend Repo Structure

```
soc-ai-rulebot-agent/
├── app.py
├── requirements.txt
├── ai_client.py
├── prompts.py
├── rule_loader.py
├── reasoning.py
├── retrieval.py
├── rag_retriever.py
├── offense_parser.py
├── handlers/
│   ├── __init__.py
│   ├── rule_handler.py
│   ├── offense_handler.py
│   └── reasoning_handler.py
├── data/
│   ├── rules/
│   ├── knowledge/
│   ├── gradar_docs/
│   └── internal_notes/
└── docs/
```

4.3 Backend Local Setup

- Clone the backend repo locally or into Cloud CLI/WSL as needed.
- Create and activate a Python virtual environment locally for development/testing.
- Install dependencies from requirements.txt.
- Set environment variables for Azure OpenAI and local runtime port before starting the app.

```
# Example local Python setup (bash style)
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Example Windows PowerShell setup
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

4.4 Backend Environment Variables

At minimum, the backend requires the following runtime variables:

```
AZURE_OPENAI_ENDPOINT=<Azure OpenAI endpoint>
AZURE_OPENAI_API_KEY=<Azure OpenAI key>
AZURE_OPENAI_CHAT_DEPLOYMENT=<chat deployment name>
```

```
AZURE_OPENAI_EMBED_DEPLOYMENT=<embedding deployment name>
AZURE_OPENAI_API_VERSION=2025-01-01-preview
PORT=8000 # or 8001 for local non-container testing
```

4.5 Backend Functional Design

- **Deterministic route:** numeric-only input routes directly to rule ID analysis for speed, low cost, and predictability.
- **Reasoning route:** free text is sent through the reasoning pipeline with retrieval-aware prompt construction.
- **Offense route:** trigger phrases such as “new offense” return a structured intake template; populated templates are parsed and analysed.
- **Rule enrichment:** rule_id is enriched from the exported QRadar rules database (individual JSON files under data/rules).
- **Retrieval:** retrieval currently supports local documents and PDFs to inject supporting context into responses.

4.6 Backend Testing Commands

Examples below assume the backend is running locally on port 8001. On Windows PowerShell use Invoke-RestMethod; on Bash use curl.

```
# PowerShell health test
Invoke-RestMethod -Uri "http://127.0.0.1:8001/health" -Method GET
```

```
# PowerShell rule test
Invoke-RestMethod -Uri "http://127.0.0.1:8001/message" -Method POST -ContentType "application/json" -Body '{
```

```
# PowerShell offense intake test
Invoke-RestMethod -Uri "http://127.0.0.1:8001/message" -Method POST -ContentType "application/json" -Body '{
```

```
# Bash/curl rule test
curl -X POST "http://127.0.0.1:8001/message" -H "Content-Type: application/json" -d '{"text":"165072"}'
```

4.7 Backend Containerization

The backend currently uses Azure Container Apps for hosting. Rebuild and redeploy are required after code changes; Git pull alone does not update the running service.

```
# Example backend image build
az acr build --registry socairulebotacr --image rulebot-agent:<tag> .
```

```
# Example backend update
az containerapp update --name soc-ai-rulebot-agent --resource-group soc-ai-rg-agents --image socairulebot-agent:<tag>
```

5. Frontend Build Guide (Current Working MVP Path)

5.1 Frontend Repository Purpose

Frontend repository: **soc-ai-rulebot-teams-frontend**. This repo contains the Teams SDK runtime that receives Teams user messages and forwards them to the backend over HTTPS.

5.2 Why a Separate Frontend Service Is Required

The Teams app ZIP is only the manifest and icons; it does not host executable code. Therefore, the Teams SDK app must run as a real HTTPS service. For production-style testing, the frontend should be deployed as its own containerized service.

5.3 Recommended Frontend Repo Structure

```
soc-ai-rulebot-teams-frontend/
■■■ app.py
■■■ requirements.txt
```

```
■■■ Dockerfile
■■■ .gitignore
■■■ README.md
■■■ .env.example
```

5.4 Frontend Local Setup

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

5.5 Frontend Runtime Logic

The frontend is intentionally thin. It receives Teams messages and forwards them to the backend endpoint. The backend does all of the actual analysis and reasoning.

```
# Frontend environment variable
BACKEND_URL=https://soc-ai-rulebot-agent.livelyocean-c3150e2c.swedencentral.azurecontainerapps.io/message
```

5.6 Frontend Containerization

The frontend should be deployed as a separate Azure Container App. Standardise on port 8000 for the container runtime.

```
# Dockerfile pattern
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV PORT=8000
EXPOSE 8000
CMD ["python", "app.py"]
```

5.7 Frontend Deployment Pattern

```
# Build frontend image
az acr build --registry socairulebotacr --image rulebot-teams-frontend:<tag> .

# Create/update frontend container app
az containerapp create --name soc-ai-rulebot-frontend --resource-group soc-ai-rg-agents --environment

# Set env vars
az containerapp update --name soc-ai-rulebot-frontend --resource-group soc-ai-rg-agents --set-env-var
```

5.8 Teams App Package / Submission Flow

- Use the Teams Developer CLI locally to create the Teams app definition and managed bot/app registration.
- Download the Teams app package (.zip) once the app has been created.
- Submit the package to IT/Teams admin for upload/approval if sideloading is blocked by policy.
- After frontend endpoint updates, regenerate the app package if required so the package points to the correct production frontend runtime.

```
# Typical Teams CLI lifecycle
teams login
teams status
teams app create --name "SOC AI Rulebot v2" --endpoint <frontend-runtime-url> --env .env
teams app list
teams app package download <APP_ID> -o SOC-AI-Rulebot-v2.zip
```

6. End-to-End MVP Test Cases

- **Rule analysis:** send 165072 in Teams -> receive rule classification/reasoning/tuning guidance.

- **Reasoning:** ask a natural-language QRadar question -> receive QRadar-focused response.
- **Offense intake:** send new offense -> receive template for analyst completion.
- **Offense analysis:** submit completed template -> receive offense analysis + tuning suggestions.

7. Why Multiple Development Paths Were Tried

This project changed path several times because earlier choices relied on product lines and integration patterns that proved brittle in practice during development. Key lessons learned:

- Older bot/channel/auth paths introduced too many independently configurable moving parts and often failed silently.
- App package approval alone does not guarantee runtime correctness if the live service endpoint and runtime model are not aligned.
- Separating frontend and backend was necessary to reach a stable architecture.
- Containerized hosting has been more reliable than app-service-style hosting attempts for this use case.
- Modern supported tooling (Teams SDK for the frontend, containerized backend API) significantly reduced uncertainty.

8. Planned Modifications Before SOC Team Release

- **Retriever quality improvements:** diversify document selection so responses do not repeatedly cite chunks from only one file.
- **RAG quality:** improve source balancing between Core_MVP and Extended_Reference docs, and refine PDF extraction quality.
- **Backend cleanup:** complete modularization into handler modules and improve maintainability.
- **Frontend hardening:** add better health/diagnostic endpoints and stabilize the production frontend deployment process.
- **Security hardening:** move toward stronger secret management and cleaner credential handling for production.
- **Operational logging:** improve frontend/backend observability and deployment runbooks.
- **Offense enrichment:** later connect offense workflows more directly with live QRadar API data and analyst context.
- **User rollout safeguards:** further internal testing before exposing the tool to the wider SOC team.

9. Recommended Production Rebuild Checklist

- Recreate backend repo cleanly from GitHub and verify local env variables.
- Rebuild backend image with a unique tag and redeploy to Azure Container Apps.
- Verify backend /health and /message endpoints over HTTPS.
- Recreate frontend repo cleanly, confirm BACKEND_URL, and rebuild frontend image with a unique tag.
- Deploy frontend as its own Azure Container App and confirm its runtime endpoint.
- Regenerate/update Teams app package so it points to the correct frontend runtime endpoint.
- Submit the package to Teams admin / IT approval flow.
- Run end-to-end tests from Teams for rule analysis, reasoning, offense intake, and offense analysis.
- Record the exact image tags, endpoint URLs, app version, and Teams app ID used for the production cut.

10. Suggested Next Technical Step

Now that the MVP works end-to-end, the most valuable next step is to improve retrieval quality so that grounded reasoning becomes more robust, more source-diverse, and more trustworthy before SOC rollout.

Appendix A — Minimal Manager Summary

The MVP is now working in Microsoft Teams using a modern supported architecture. Earlier difficulties were caused by unstable or deprecated integration paths rather than by the core Rulebot logic itself. The backend service is functioning and reusable, and the Teams-facing frontend path has now been modernized. Remaining work before broader SOC release focuses on quality and hardening, not on proving the concept.

Appendix B — File/Repo Notes

- **Backend repo:** intelligence, prompts, retrieval, offense logic, rule data, deployment to backend container app.
- **Frontend repo:** Teams SDK runtime only, direct HTTPS call to backend, packaging for Teams app submission.
- **Teams app package ZIP:** distribution artifact only; should not be committed to Git.
- **.venv:** local developer-only; excluded from source control.